

GitHub Actions

Automatisez vos workflows de build, test et déploiement
directement depuis votre dépôt GitHub.

SOMMAIRE

01 Introduction & Concepts clés

02 Structure d'un Workflow

03 Déclencheurs (on:)

04 Jobs, Steps & Dépendances

05 Variables & Secrets

06 Matrix Strategy

07 Artifacts & Cache

08 Déploiement (CD)

09 Bonnes Pratiques

01 Introduction & Concepts clés

GitHub Actions est une plateforme d'automatisation intégrée à GitHub. Elle permet de créer des pipelines CI/CD, d'automatiser des tâches de maintenance, ou de réagir à n'importe quel événement du cycle de vie d'un dépôt.

 Workflow Processus automatisé défini dans un fichier YAML dans <code>.github/workflows/</code> .	 Event Déclencheur du workflow : <code>push</code> , <code>pull_request</code> , <code>schedule</code> , etc.	 Job Ensemble de steps s'exécutant sur un même runner (machine virtuelle).
 Step Tâche individuelle : exécuter un script ou appeler une action.	 Runner Machine (ubuntu, windows, macos) qui exécute les jobs.	 Action Unité réutilisable publiée sur le Marketplace GitHub.

02 Structure d'un Workflow

Tout workflow est un fichier `.yaml` placé dans `.github/workflows/`. Voici la structure minimale complète :

```
YAML                                                                    .github/workflows/ci.yaml

name: CI Pipeline

on:                                # Déclencheurs
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest # Type de runner

    steps:
      - name: Checkout du code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Installer les dépendances
        run: npm ci

      - name: Lancer les tests
        run: npm test
```

03 Déclencheurs (on:)

La clé `on` définit quand le workflow se déclenche. Voici les événements les plus utilisés :

Événement	Description	Exemple
<code>`push`</code>	À chaque push	Sur <code>`main`</code> uniquement
<code>`pull_request`</code>	Ouverture/MAJ d'une PR	Vers <code>`main`</code>
<code>`schedule`</code>	Planification cron	Chaque nuit à minuit
<code>`workflow_dispatch`</code>	Déclenchement manuel	Via l'UI GitHub

`release`	Publication d'une release	Type `published`
`workflow_call`	Appelé par un autre workflow	Workflows réutilisables

YAML

Déclencheurs avancés

```

on:
  schedule:
    - cron: '0 0 * * *'      # Chaque jour à minuit UTC

  workflow_dispatch:         # Déclenchement manuel
    inputs:
      environment:
        description: 'Env cible'
        required: true
        default: 'staging'
        type: choice
        options: [staging, production]

  push:
    paths:                    # Seulement si ces fichiers changent
      - 'src/**'
      - 'package.json'
    tags:
      - 'v*'                  # Seulement sur les tags de version

```

04 Jobs, Steps & Dépendances

Les jobs s'exécutent en **parallèle** par défaut. Utilisez **needs** pour créer des dépendances et définir un ordre d'exécution.

YAML

Pipeline avec dépendances

```

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test

  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run lint

  build:
    needs: [test, lint]      # S'exécute après test ET lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build

  deploy:
    needs: build
    if: github.ref == 'refs/heads/main' # Condition
    runs-on: ubuntu-latest
    steps:
      - run: echo 'Déploiement en production !'

```

■ **Astuce:** Utilisez `if: always()` pour qu'un job s'exécute même si les jobs précédents échouent. Utile pour les rapports et notifications.

05 Variables & Secrets

GitHub Actions dispose de plusieurs niveaux de variables : les variables d'environnement, les secrets chiffrés, et les contextes pré-définis.

```
env:                                # Variables globales au workflow
  NODE_ENV: production
  APP_VERSION: '1.0.0'

jobs:
  deploy:
    runs-on: ubuntu-latest
    env:      # Variables locales au job
      DEPLOY_ENV: staging

    steps:
      - name: Déployer
        env:  # Variables locales au step
          API_KEY: ${ secrets.API_KEY }
          DB_URL:  ${ secrets.DATABASE_URL }
        run: |
          echo 'Déploiement sur $DEPLOY_ENV'
          ./deploy.sh --env $DEPLOY_ENV

      - name: Infos contextuelles
        run: |
          echo 'Branche : ${ github.ref_name }'
          echo 'SHA      : ${ github.sha }'
          echo 'Acteur   : ${ github.actor }'
```

■ Sécurité

: Ne jamais écrire de secrets directement dans les fichiers YAML. Utilisez Settings → Secrets and variables → Actions dans GitHub pour les stocker.

06

Matrix Strategy

La stratégie matrix permet d'exécuter un même job avec différentes combinaisons de paramètres, idéale pour tester sur plusieurs versions ou OS.

```
jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      fail-fast: false          # Continuer même si un job échoue
    matrix:
      os: [ubuntu-latest, windows-latest, macos-latest]
      node: ['18', '20', '22']
      exclude:                  # Exclure certaines combinaisons
        - os: windows-latest
          node: '18'

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node }
      - run: npm ci && npm test
```

07

Artifacts & Cache

Les **artifacts** permettent de partager des fichiers entre jobs ou de les conserver après le workflow. Le **cache** accélère les builds en réutilisant les dépendances.

YAML

Cache des dépendances

```
steps:
  - uses: actions/checkout@v4

  - name: Cache node_modules
    uses: actions/cache@v4
    with:
      path: ~/.npm
      key: ${ runner.os }-node-${ hashFiles('**/package-lock.json') }
      restore-keys: |
        ${ runner.os }-node-

  - run: npm ci && npm run build

  - name: Upload artifact
    uses: actions/upload-artifact@v4
    with:
      name: dist-files
      path: dist/
      retention-days: 7
```

YAML

Télécharger un artifact dans un autre job

```
deploy:
  needs: build
  steps:
    - name: Download artifact
      uses: actions/download-artifact@v4
      with:
        name: dist-files
        path: ./dist
```

08

Déploiement (CD)

Exemple complet de pipeline CI/CD pour une application Node.js déployée sur un VPS via SSH :

YAML

.github/workflows/deploy.yml

```

name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci && npm test && npm run build
      - uses: actions/upload-artifact@v4
        with: { name: build, path: dist/ }

  deploy:
    needs: ci
    runs-on: ubuntu-latest
    environment: production    # Requiert une approbation manuelle

    steps:
      - uses: actions/download-artifact@v4
        with: { name: build, path: dist/ }

      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.SERVER_HOST }
          username: ${ secrets.SERVER_USER }
          key: ${ secrets.SSH_PRIVATE_KEY }
          script: |
            cd /var/www/app
            git pull origin main
            npm ci --production
            pm2 restart app

```

09

Bonnes Pratiques

■ Épinglez

les versions d'actions — Préférez actions/checkout@v4 ou même le SHA exact (@abc123f) plutôt que @main pour éviter les changements inattendus.

■ Utilisez

le cache — Mettez toujours en cache node_modules, le cache pip ou Maven pour réduire considérablement les temps de build.

■ Principe

du moindre privilège — Limitez les permissions des tokens avec permissions: read-all au niveau global.

■ Évitez

pull_request_target — Cet événement accède aux secrets depuis les forks, ce qui peut exposer vos credentials si le code de la PR est malveillant.

YAML

Permissions minimales recommandées

```

permissions:
  contents: read
  pull-requests: write    # Seulement si nécessaire
  packages: write        # Seulement si nécessaire

```



Réutilisabilité

Créez des workflows réutilisables avec workflow_call pour éviter la duplication.



Nommage clair

Donnez des noms explicites à vos jobs et steps pour faciliter le débogage.



Environnements

Utilisez les Environments GitHub pour les approbations de déploiement.



Timeouts

Définissez `timeout-minutes` pour éviter les jobs bloqués qui consomment des minutes.



Concurrency

Utilisez `concurrency`: pour annuler les anciennes exécutions lors de nouveaux pushes.



Observabilité

Ajoutez des `step summaries` avec `$GITHUB_STEP_SUMMARY` pour des rapports visuels.